

# S2PD: SERIAL-TO-PARALLEL DIFFUSION FOR PHYSICALLY AND LOGICALLY CONSISTENT VIDEO GENERATION

Jeffrey Hu<sup>1</sup> Daniel Olmeda Reino<sup>2</sup> Ayush Tewari<sup>1</sup>

<sup>1</sup>University of Cambridge <sup>2</sup>Toyota Motor Europe

## ABSTRACT

Bidirectional video diffusion models denoise entire videos in parallel, yet when trained on effectively unlimited in-distribution data from procedural generators, continue to violate physical laws and simple symbolic rules. We introduce Serial-to-Parallel Diffusion (S2PD), which performs autoregressive diffusion at high noise before switching to parallel diffusion at low noise. The autoregressive phase provides the serial computation needed to coordinate interdependent events and produce valid state transitions while the parallel phase jointly refines the entire video and reduces sampling time relative to fully serial generation. We implement S2PD with two architectures: a pixel-space diffusion transformer trained from scratch and a pretrained video model adapted through LoRA fine-tuning with causal attention. Across games, physical simulations, and real video, S2PD follows rules more reliably than matched bidirectional baselines and generates videos with greater temporal stability and sampling efficiency than other serial methods.

## 1 INTRODUCTION

Bidirectional diffusion transformers such as Wan (Wan Team, 2025) are widely used for large-scale video generation. These models denoise all video tokens in parallel. Although these models achieve high visual fidelity as measured by FVD (Unterthiner et al., 2018) and VBench (Huang et al., 2024), multiple studies document morphing artifacts and violations of physical laws in their samples (Bansal et al., 2026; Motamed et al., 2026). We discover a more puzzling result: bidirectional video diffusion transformers continue to exhibit these failures even when trained on effectively unlimited procedurally generated data from relatively simple simulations. Physical accuracy therefore does not emerge from data scaling alone.

The Serial Scaling Hypothesis (Liu et al., 2026) offers a theoretical explanation. Liu et al. prove that diffusion models remain in  $TC^0$ , a class of computations with bounded sequential depth, even with arbitrarily many denoising steps. For standard diffusion models, increasing model depth is the only way to supply the serial computation some problems require. Videos can contain long chains of causally dependent events. We hypothesize that bidirectional diffusion models, early in denoising when the video’s overall structure is ambiguous, assign locally plausible but mutually incompatible events to different space–time regions. Subsequent denoising sharpens these commitments without reconciling them, leading to abrupt changes or invalid transitions.

Our solution is to let the model commit to structure serially by generating blocks of video tokens autoregressively. Concurrent work on the Seriality Gap (Díaz Chao et al., 2026) shows that autoregressive generation with increasingly small block sizes reduces errors on colliding ball simulations. We implement two autoregressive diffusion methods and demonstrate a similar pattern across a broad range of games and simulations. We further adapt a large pretrained bidirectional video model for autoregressive generation through LoRA fine-tuning (Hu et al., 2022) on more complex simulations and real video. On real video of a Rubik’s Cube, our models generate realistic hand–cube interactions while maintaining a plausible cube state over time.

Finally, we introduce Serial-to-Parallel Diffusion (S2PD). Although autoregressive diffusion models follow rules much better than bidirectional diffusion models, they are slow to sample and susceptible

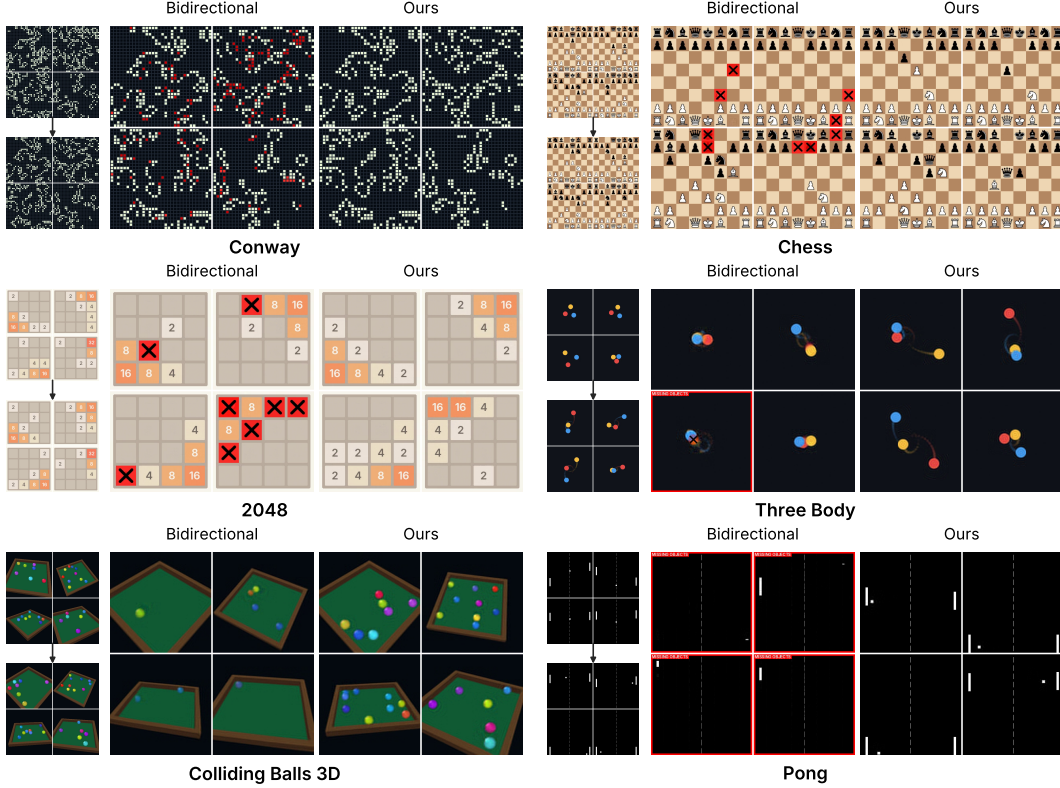


Figure 1: **Serial outperforms parallel generation.** Our method better preserves game rules, physical dynamics, and object consistency across all datasets. Red overlays localize errors in the generated samples.

to temporal instability and autoregressive drift (Huang et al., 2025). Autoregressive drift occurs when generated content is used as context for generating more content later. Small prediction errors accumulate, causing sampling and training distributions to diverge. S2PD divides sampling into two phases, confining the autoregressive diffusion essential for logical consistency to the drift-resistant high-noise regime, before refining the full sequence in parallel at low noise.

Our contributions are as follows:

- We introduce S2PD, which preserves the physical plausibility and logical consistency of serial generation while improving sampling efficiency and temporal stability.
- We show that bidirectional diffusion violates physical and symbolic rules despite effectively unlimited procedurally generated in-distribution data, whereas autoregressive diffusion methods substantially reduce these errors.
- We provide datasets of games and simulations, along with automated evaluation metrics for rule violations and physical accuracy. We have released all code, datasets, and model weights.

## 2 RELATED WORK

**Autoregressive generation.** Video Diffusion Models (Ho et al., 2022) extends image diffusion (Ho et al., 2020) to jointly denoise all video tokens in parallel. In contrast, autoregressive video models generate tokens sequentially, using classification for discrete tokens (Yan et al., 2021) or diffusion for continuous tokens. Diffusion Forcing (Chen et al., 2024) achieves autoregressive video diffusion by training a model to denoise independently noised frames. Self Forcing (Huang et al., 2025) points out that repeated autoregressive denoising leads to a training–inference distribution mismatch and addresses this by also training on generated video. Block Diffusion (Arriola

et al., 2025) introduces a method for generating chunks of text simultaneously to accelerate language modeling. We find block-causal diffusion particularly well suited to video modeling since it can generate whole frames at once. Flex-Forcing (Ma et al., 2026) uses block-causal diffusion with variable block sizes, but they only explore block sizes larger than one frame and use it as an acceleration technique with efficiency–quality trade-offs. We explore block sizes smaller than one frame and study how spatio-temporal autoregression affects logical consistency.

**World modeling.** Oh et al. (2015) demonstrate action-conditioned video generation in Atari games. World models such as Genie 3 (Google DeepMind, 2025) and Cosmos 3 (NVIDIA, 2026) have scaled this to richer environments. These models are naturally autoregressive to allow users to submit actions that affect the generated video stream in real time. Thus, they already support the serial computation necessary for logical consistency. MIRA (Hu et al., 2026) demonstrates remarkable physics consistency in the highly dynamic game of Rocket League. Our work explains the role of serial computation in achieving such high fidelity and shows that video models can achieve strong physical and logical consistency without a temporally dense action-conditioning signal. World models may also benefit from a parallel refinement stage.

Frontier coding models such as GPT-6 Astra (OpenAI, 2026) and Claude Fable 5 (Anthropic, 2026) can also reconstruct scenes and run simulations through code and software tools. VDA-World (O’Mahony et al., 2026) and VIGA (Yin et al., 2026) construct executable programs to represent scenes explicitly before rendering them. VLMs supply the required serial computation through text. We achieve it through autoregressive video generation without relying on human-labeled expert trajectories.

**Evaluating physical consistency.** Numerous benchmarks such as Physics-IQ (Motamed et al., 2026), PhyWorldBench (Gu et al., 2026), T2VPhysBench (Guo et al., 2025), and PhyGround (Lin et al., 2026) test whether video models correctly capture physical laws and dynamics. Although these benchmarks remain unsaturated, their real-world setting and generalization demands make it difficult to isolate the causes of failure. We study simpler games and simulations, where video parsing provides quantitative feedback on rule violations and physical error, before scaling up to more complex datasets.

### 3 METHOD

#### 3.1 BACKGROUND

We adopt the modern flow-matching (Lipman et al., 2023) formulation of diffusion. Let  $x_0$  denote a clean target. We corrupt  $x_0$  with Gaussian noise  $\epsilon \sim \mathcal{N}(0, I)$  along a linear path

$$x_t = (1 - t)x_0 + t\epsilon, \quad (1)$$

where  $t = 0$  denotes clean data and  $t = 1$  denotes noise. Following JiT (Li & He, 2026), we use  $x$ -prediction with  $v$ -loss: the model predicts the clean target  $\hat{x}_0 = f_\theta(x_t, t)$ , while the loss is computed with mean squared error in velocity space. We integrate the velocity  $\hat{v}_\theta = (x_t - \hat{x}_0)/t$  from  $t = 1$  to  $t = 0$  during sampling.

For video generation, we patchify the video into *tubelets*: spatial patches of size  $p_h \times p_w$  spanning  $p_t$  consecutive frames. Each tubelet becomes one transformer token. Smaller  $p_t$  provides finer temporal granularity at the cost of increased token count, which increases training and inference times. Inspired by JiT’s use of large image patches, we send the full  $3p_t p_h p_w$ -dimensional tubelet through our model. We use a linear bottleneck before projecting to the transformer width.

**Token order and block size.** When flattening a 3D video token grid into a sequence, we first define a consistent token order and use 3D RoPE to retain information about the tokens’ spatial and temporal positions after flattening. See Section 5.1 for the token order ablation. We then group consecutive tokens into  $K$  blocks  $B^0, \dots, B^{K-1}$  of block size  $b$ . Tokens within a block share a noise level during training and are denoised together during inference. Smaller blocks allow more autoregressive steps during sampling. When the block size equals the total sequence length, generation becomes equivalent to full-video bidirectional diffusion.

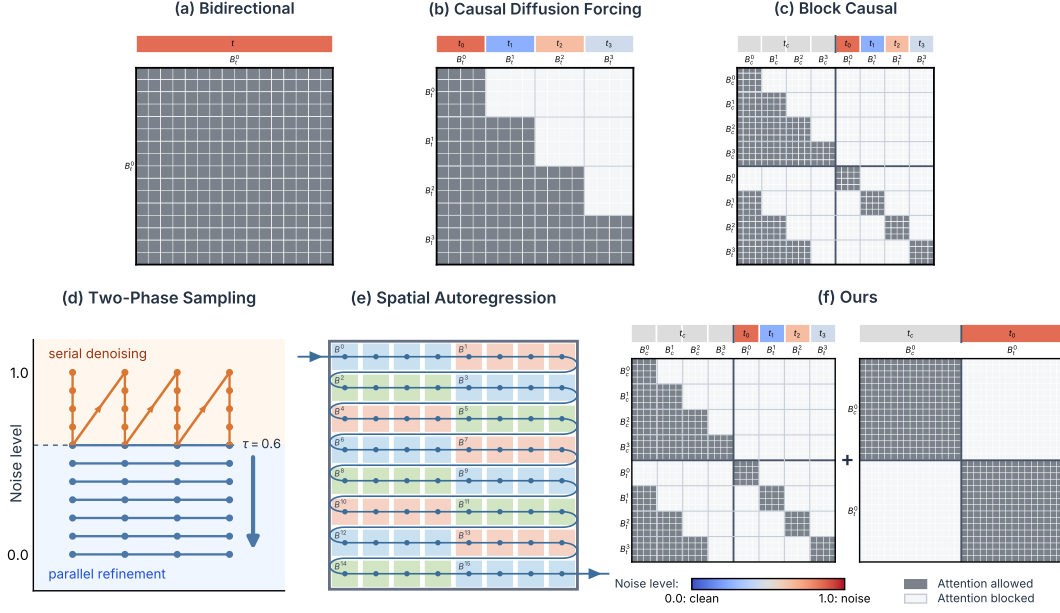


Figure 2: **Attention patterns across methods and our sampling strategy.** (a–c) Attention masks and noise levels for Bidirectional, cDF, and BC. (d) Serial denoising followed by parallel refinement. (e) Token order within a temporal slice. (f) Multiple block sizes used by Ours during training.

**Attention masks and noise levels.** As shown in Figure 2a, bidirectional diffusion uses standard self-attention and one noise level for the entire video. Each video token can attend to all video tokens during training and the entire sequence is denoised in parallel during inference. As shown in Figure 2b, Diffusion Forcing (Chen et al., 2024) uses independent noise levels for each block during training. This enables autoregressive generation because the model learns to denoise each block using the surrounding partially noised context. We implement a causal version of Diffusion Forcing Transformer (Song et al., 2025), which we refer to as cDF, to better suit autoregressive generation. We replace the default self-attention with block-causal attention. Block-causal attention restricts each block to attend only to itself and preceding blocks. As shown in Figure 2c, block-causal diffusion (Arriola et al., 2025), which we refer to as BC, uses two copies of the token sequence: a context copy with one noise level  $t_c$  and a target copy with independent noise levels  $t_k$ . We denote context and target blocks,  $B_c^k$  and  $B_t^k$ , respectively. Each context block attends to itself and preceding context blocks. Each target block attends to itself and preceding context blocks, but not other target blocks. Loss is computed only on the target blocks.

### 3.2 SAMPLING

S2PD generates blocks autoregressively at high noise before refining the whole video together in parallel at low noise. As shown in Figure 2d, we define a transition noise level  $\tau \in [0, 1]$  to separate the two phases. By default, we use ten uniformly spaced denoising steps and  $\tau = 0.6$  to allocate four denoising steps to serial denoising and six to parallel refinement. Let  $N_f$  denote the number of tokens in one temporal slice of the video token grid and  $N_v$  the number of tokens in the full video. We use a block size  $b \leq N_f$  for serial denoising and  $b = N_v$  for parallel refinement.

**Serial denoising.** We initialize blocks of size  $b$  with Gaussian noise and denoise them autoregressively from  $t = 1$  to  $t = \tau$ . Each block is conditioned on a key–value (KV) cache of previously generated blocks at noise level  $\tau$ . We follow the token order shown in Figure 2e. For cDF and BC, we always use a block size of  $N_f$  and fully denoise each block to  $t = 0$ . We re-noise the block before adding it to the KV cache to mitigate autoregressive drift, using context noise level  $t_c = 0.2$  as default.

**Parallel refinement.** For S2PD, once all blocks reach  $\tau$ , we treat the full noisy video as a single block and denoise it from  $t = \tau$  to  $t = 0$ . Despite using the same number of denoising steps per token, parallel refinement is much faster than serial denoising because all tokens are batched together. This amortizes the cost of reading model weights from memory. Sampling is usually limited by memory bandwidth, not compute.

### 3.3 TRAINING

S2PD learns to denoise each target block using only the preceding context blocks at noise level  $t_c$  as context. Setting  $t_c = \tau$  would match the context noise level used during training with the transition noise level used during sampling. By default, we train with  $t_c \sim \mathcal{U}(0, 1)$  to support different transition noise levels at inference time. We sample the noise level for each target block  $t_k$  from a plateau logit-normal distribution (Black Forest Labs, 2025). This distribution preserves training coverage at high noise levels.

As shown in Figure 2f, S2PD uses the same token sequence, attention pattern, and noise levels as block-causal diffusion. The only difference is that we train with multiple block sizes. We mix samples with different block sizes within the same training batch. Training with the block size mixture  $b \in \{N_f, N_v\}$  would support S2PD sampling at a single block size. We use  $b \in \{4, 8, \dots, N_f, N_v\}$  to support flexible block sizes. We delay the choice of both block size and transition noise level to inference time. To support image-to-video generation, we prepend the clean condition-image tokens to the token sequence. The image tokens are omitted from the attention patterns in Figure 2 for clarity.

We implement all four methods—Bidirectional, cDF, BC, and S2PD—with two architectures: DiT-B and Wan-5B-LoRA. DiT-B is a 132.7M-parameter diffusion transformer (Peebles & Xie, 2023) trained from scratch in pixel space. Wan-5B-LoRA is a pretrained latent-space video diffusion model adapted using rank-32 LoRA (Hu et al., 2022) with 80.6M trainable parameters in total. In Wan-5B, we replace the default attention mask with each method’s custom attention pattern and let the adapters adapt. Since BC and S2PD use double the sequence length during training, we double the batch size used by Bidirectional and cDF to compensate. This matches training token budgets and roughly matches total training FLOPs. For more implementation details, see Appendix A.

## 4 DATASETS

Our dataset collection spans games, simulations and real video. For simple datasets, we implement video parsing and symbolic evaluation to detect rule and physics violations. For more complex datasets, we rely on distributional metrics based on Fréchet distance (Dowson & Landau, 1982).

### 4.1 DATASET COLLECTION

First, we implement Conway’s Game of Life (Gardner, 1970), a zero-player game where a grid of live and dead cells evolves deterministically. At each step, all cells update simultaneously according to their neighbors. A live cell survives if it has two or three live neighbors and dies otherwise. A dead cell comes to life if it has exactly three live neighbors. To create our video dataset, we hold each step for  $p_t$  frames, aligning state transitions with token boundaries.

Next, we add other discrete datasets like Chess, 2048, 15 Puzzle, Tetris, Snake, and Rubik’s Cube 3D. We animate transitions between states to make the dynamics easier for a video model to learn. Animations occupy the first  $p_t - 1$  frames of each temporal block but are not used during evaluation. We parse only the settled frame at the end of the state transition. If the game ends early, we overlay an end-of-game modal and retain it for the rest of the video.

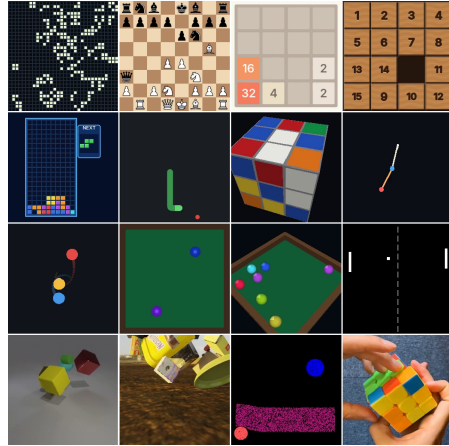


Figure 3: **Samples from all datasets.** Games, simulations, and real video.

Then, we add continuous simulations to test physical dynamics. We implement Double Pendulum, three-body orbital dynamics, 2D and 3D colliding balls, and Pong. Since we evaluate for valid transitions rather than optimal play for games, we either sample random legal moves or use bots to produce reasonable move sequences. For Chess, we downloaded 1M human-played chess games from Lichess. All datasets so far are rendered on-the-fly during training.

Lastly, we include datasets with more complex simulations and real video. These datasets are pre-generated offline, downloaded with their official splits, or collected ourselves. We use Kubric MOVi-A and MOVi-C (Greff et al., 2022) to simulate 3D colliding objects

at two different levels of complexity. We add MPMWorlds (Kovačič & Ellis, 2026), which uses 2D material point method simulation with deformable objects, fluids, and moving obstacles to test dynamics involving deformation and fluid flow. We collect Rubik’s Cube Real, a dataset containing 30 minutes of real footage of scrambling and solving a Rubik’s Cube. We divide the footage into non-overlapping training and validation segments, then extract heavily overlapping 10-second clips within each split.

Table 1: **Dataset pool sizes.** All videos are 10 seconds at 24 fps and  $256 \times 256$ .

| Dataset                    | Train pool    | Val pool      |
|----------------------------|---------------|---------------|
| <i>Discrete datasets</i>   |               |               |
| All                        | $\sim \infty$ | $\sim \infty$ |
| <i>Continuous datasets</i> |               |               |
| All                        | $\sim \infty$ | $\sim \infty$ |
| <i>Video datasets</i>      |               |               |
| Kubric MOVi-A              | 9,703         | 250           |
| Kubric MOVi-C              | 9,737         | 250           |
| MPMWorlds                  | 9,439         | 1,017         |
| Rubik’s Cube Real          | 5,000         | 512           |

## 4.2 VIDEO PARSING AND EVALUATION

For simple datasets, we evaluate generated videos by parsing them into symbolic states and checking whether their transitions obey the underlying game rules or physical dynamics. Discrete datasets evolve through a finite set of transitions on a fixed grid. Since the grid positions are known ahead of time, we parse their videos through template matching of their grid cells. Then, we evaluate each pair independently, checking state validity and the permitted changes between frames. Persistent invalid states incur repeated penalties. We count *invalid transitions per rollout* as our primary metric. For Rubik’s Cube 3D, since the camera pose is unknown, we fit a 3D cube to the 2D silhouette before parsing the facelet colors. For continuous datasets, we track expected objects and fit simulation parameters. Then, we account for occlusions and measure deviations from the expected dynamics. We report cumulative error as *dynamics error per rollout*. We use red diagnostic overlays to localize the errors we detect and verify each dataset evaluation procedure manually on generated samples. We invite the reader to use our project page to do the same. Parsing and tracking failures complicate our evaluation metrics. For more details, see Appendix B.

With these metrics, Figure 4 shows that the bidirectional baseline never learns to generate valid transitions for Conway over the course of training. In contrast, our method rapidly learns the rules and generates videos with almost no errors. Our video parsing and metrics do not require a pixel-aligned ground-truth video for evaluation. We evaluate each sample against the underlying rules and dynamics of the simulator. Figure 5 shows that our method produces diverse valid continuations from a single image. However, for video datasets that are too complex to parse reliably, we fall back to using distributional metrics like FVD. We adopt CD-FVD (Ge et al., 2024) as our primary distributional metric because it uses a more modern feature extractor and has been shown to have less bias toward per-frame appearance and greater sensitivity to temporal distortions than FVD. Nonetheless, we also report FVD in Appendix C.2.

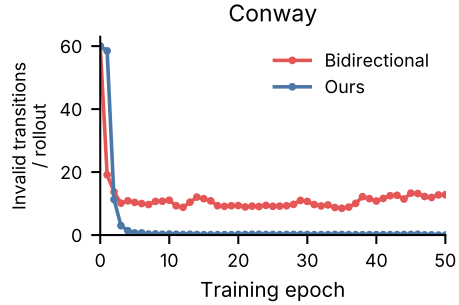


Figure 4: **Learning Conway rules.** Ours rapidly learns valid transitions, while the baseline continues to make errors.

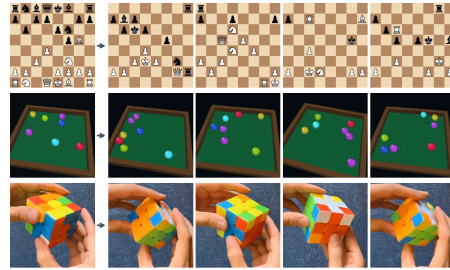


Figure 5: **One image, multiple continuations.** Ours produces multiple plausible continuations from the same image.

Table 2: **Main comparisons.** Serial methods outperform the bidirectional baseline and perform similarly to each other across all datasets. cDF and BC use block sizes of 64. Ours uses a block size of 8 for discrete datasets and 64 for continuous and video datasets. Best is marked in blue and worst is marked in red.

| Dataset                    | Architecture | Metric                                  | Parallel      | Serial |       |       |
|----------------------------|--------------|-----------------------------------------|---------------|--------|-------|-------|
|                            |              |                                         | Bidirectional | cDF    | BC    | Ours  |
| <i>Discrete datasets</i>   |              |                                         |               |        |       |       |
| Conway                     | DiT-B        | <u>Invalid transitions</u><br>rollout ↓ | 9.4           | 0.5    | 0.0   | 0.0   |
| Chess                      |              |                                         | 51.0          | 4.6    | 6.8   | 12.3  |
| 2048                       |              |                                         | 47.9          | 1.5    | 1.4   | 0.9   |
| Fifteen Puzzle             |              |                                         | 59.1          | 21.0   | 7.3   | 12.7  |
| Tetris                     |              |                                         | 57.2          | 4.5    | 5.3   | 3.2   |
| Snake                      |              |                                         | 47.2          | 1.4    | 1.4   | 2.4   |
| Rubik’s Cube 3D            |              |                                         | 46.1          | 26.5   | 31.1  | 22.0  |
| <i>Continuous datasets</i> |              |                                         |               |        |       |       |
| Double Pendulum            | DiT-B        | <u>Dynamics error</u><br>rollout ↓      | 73.4          | 1.2    | 1.2   | 1.1   |
| Three Body                 |              |                                         | 59.5          | 0.4    | 0.2   | 0.6   |
| Colliding Balls 2D         |              |                                         | 5.3           | 0.0    | 0.0   | 0.0   |
| Colliding Balls 3D         |              |                                         | 6.3           | 0.1    | 0.1   | 0.0   |
| Pong                       |              |                                         | 159.7         | 0.1    | 0.2   | 1.3   |
| <i>Video datasets</i>      |              |                                         |               |        |       |       |
| Kubric MOVi-A              | Wan-5B-LoRA  | CD-FVD ↓                                | 252.8         | 168.5  | 141.2 | 137.0 |
| Kubric MOVi-C              |              |                                         | 512.4         | 376.5  | 240.3 | 231.7 |
| MPMWorlds                  |              |                                         | 88.7          | 43.2   | 43.9  | 42.5  |
| Rubik’s Cube Real          |              |                                         | 698.7         | 70.8   | 70.9  | 64.4  |

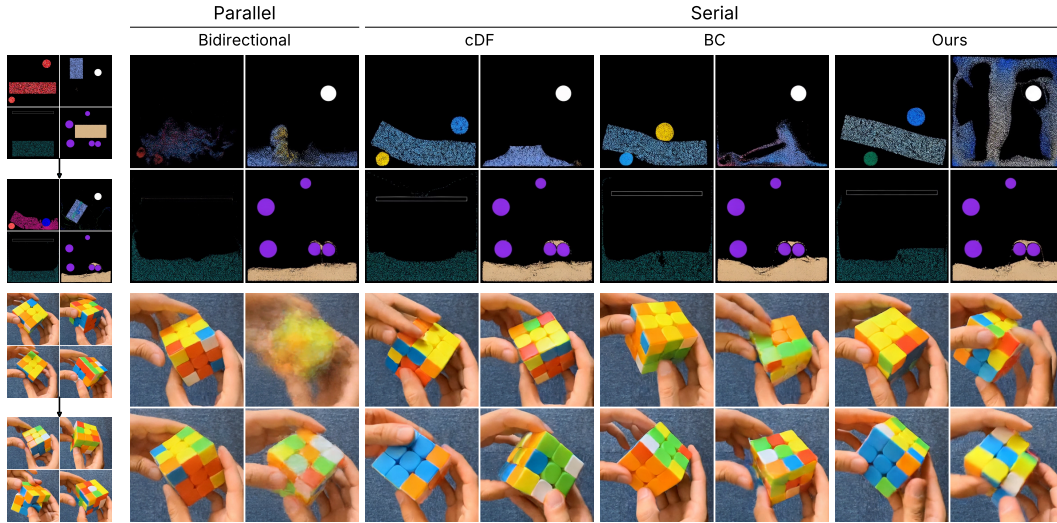


Figure 6: **Qualitative comparisons with the Wan-5B-LoRA architecture.** Serial methods perform similarly and produce more coherent video than the bidirectional baseline on the MPMWorlds and Rubik’s Cube Real datasets.

## 5 RESULTS

We train a dedicated model for each dataset and method. As shown in Table 2, all serial methods perform similarly on discrete and continuous datasets with no clear winner. However, they dramatically outperform the bidirectional baseline. Figure 1 shows still images from video samples generated by our method and the baseline. Diagnostic overlays highlight errors in red. The baseline frequently makes invalid chess moves and 2048 transitions. It loses balls in simulations and produces unrealistic Pong gameplay. While not perfect, our method, and all serial methods in fact, generate much more coherent video than the baseline, with the same model size. We show uncurated samples for all

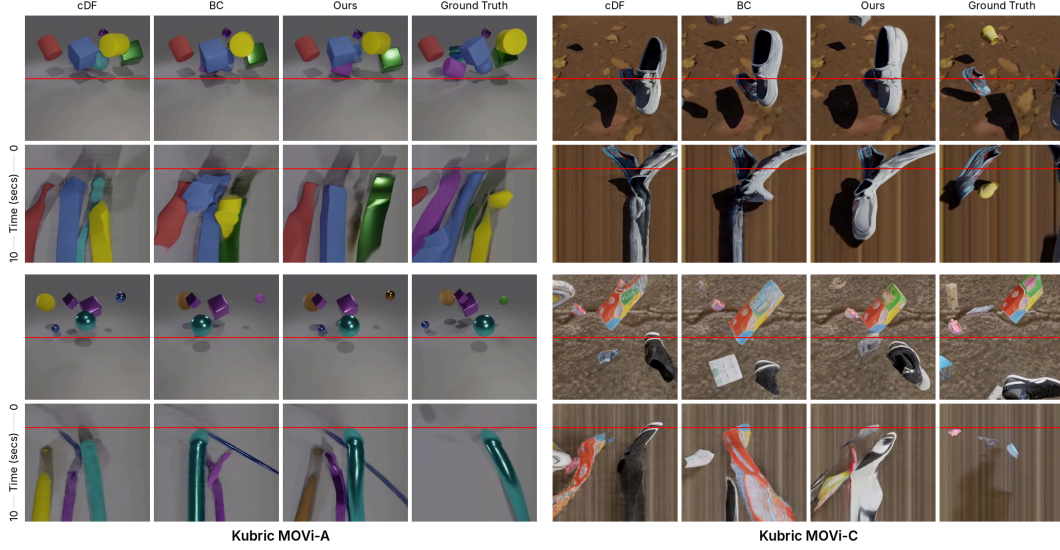


Figure 7: **Spatiotemporal slices expose temporal flickering.** Ours and ground truth show clean vertical lines while cDF and BC show subtle patchy artifacts.

methods in the video gallery on our project page. Serial generation continues to outperform the parallel baseline on more complex datasets with a larger architecture. Figure 6 shows that the standard configuration of LoRA on Wan-5B generates implausible material deformations on MPMWorlds. Matched serial counterparts generate much more reasonable continuations and perform much better on Rubik’s Cube Real.

**Sampling efficiency.** With the same block size, Table 3 shows that S2PD achieves the lowest CD-FVD at twice the speed compared to other serial methods. Table 4 shows that for S2PD, a transition noise level  $\tau = 0.6$  is more than twice as fast as  $\tau = 0.0$  across all block sizes.

**Temporal stability.** For Kubric datasets, purely serial generation shows temporal instability or flicker.

This is best viewed on the project website. Figure 7 shows  $x-t$  spatiotemporal slices formed by stacking the horizontal center line of each frame vertically. Our method and ground truth show clean vertical lines, particularly in the static background, whereas BC and cDF show subtle patchy artifacts. Our parallel refinement stage lets all blocks reconcile their appearance, smoothing artifacts and achieving the lowest CD-FVD on all four video datasets.

## 5.1 ABLATIONS

We vary the transition noise level, block size, and token order to test how the amount and organization of serial computation affect generation quality and sampling time. Unless specified otherwise, all ablations use DiT-B models trained from scratch with block sizes  $b \in \{4, 8, \dots, N_f, N_v\}$  and sampled with a block size  $b = 8$  and a transition noise level  $\tau = 0.6$ .

**Transition noise level.** The transition noise level  $\tau$  determines how much denoising is performed serially before parallel refinement begins. We can vary it at inference time from fully parallel generation at  $\tau = 1.0$  to fully serial generation at  $\tau = 0.0$ . However,  $\tau = 0.0$  is not recommended because, at this setting, the model is highly susceptible to autoregressive drift. Table 4 shows that  $\tau = 0.6$  achieves a similar score to  $\tau = 0.2$  while being significantly faster. We use  $\tau = 0.6$  as the

Table 3: **CD-FVD ↓ with Wan-5B-LoRA at block size 64.** Sampling times are shown in parentheses.

|                       | cDF            | BC             | Ours          |
|-----------------------|----------------|----------------|---------------|
| Block size            | 64             | 64             | 64            |
| <i>Video datasets</i> |                |                |               |
| Kubric MOVi-A         | 168.5 (13.8 s) | 141.2 (13.7 s) | 137.0 (6.8 s) |
| Kubric MOVi-C         | 376.5 (13.7 s) | 240.3 (13.8 s) | 231.7 (6.8 s) |
| MPMWorlds             | 43.2 (13.7 s)  | 43.9 (13.6 s)  | 42.5 (6.9 s)  |
| Rubik’s Cube Real     | 70.8 (13.8 s)  | 70.9 (13.7 s)  | 64.4 (7.0 s)  |

Table 4: **Balancing serial denoising and parallel refinement.** Moderate transition noise levels perform similarly. Small blocks improve performance but increase sampling cost. We report *invalid transitions per rollout*  $\downarrow$  with sampling time in parentheses. Default settings are marked in grey.

| Block size            |     | Chess        |              |               |               | 2048         |              |              |              |
|-----------------------|-----|--------------|--------------|---------------|---------------|--------------|--------------|--------------|--------------|
|                       |     | 64           | 32           | 16            | 8             | 64           | 32           | 16           | 8            |
| <i>fully parallel</i> | 1.0 | 52.1 (0.1 s) | 52.1 (0.1 s) | 52.1 (0.1 s)  | 52.1 (0.1 s)  | 48.5 (0.1 s) | 48.5 (0.1 s) | 48.5 (0.1 s) | 48.5 (0.1 s) |
|                       | 0.8 | 31.7 (1.0 s) | 28.3 (1.9 s) | 17.1 (3.6 s)  | 15.3 (6.4 s)  | 11.4 (0.9 s) | 8.3 (1.7 s)  | 4.7 (3.3 s)  | 3.1 (5.4 s)  |
| $\tau = 0.6$          | 0.6 | 24.6 (1.6 s) | 24.6 (3.0 s) | 15.1 (5.9 s)  | 13.3 (10.4 s) | 4.5 (1.5 s)  | 3.0 (2.9 s)  | 1.3 (5.7 s)  | 1.0 (9.3 s)  |
|                       | 0.4 | 24.0 (2.2 s) | 24.8 (4.2 s) | 15.1 (8.3 s)  | 13.9 (13.4 s) | 4.4 (2.1 s)  | 2.7 (4.0 s)  | 1.5 (7.9 s)  | 1.0 (12.7 s) |
|                       | 0.2 | 24.6 (2.8 s) | 23.5 (5.4 s) | 14.1 (10.7 s) | 13.1 (17.2 s) | 4.3 (2.7 s)  | 2.7 (5.1 s)  | 1.6 (10.1 s) | 1.1 (16.5 s) |
| <i>fully serial</i>   | 0.0 | 34.0 (3.4 s) | 32.5 (6.5 s) | 27.1 (13.0 s) | 26.3 (21.0 s) | 6.3 (3.1 s)  | 5.5 (6.1 s)  | 3.3 (12.0 s) | 2.7 (19.2 s) |

Table 5: **Block size ablation.** Decreasing block size reduces errors more for discrete datasets than continuous datasets. Training for a specific block size mostly improves discrete datasets.

| Block size                 | Pin  | Flex |      |      |      | Pin  |
|----------------------------|------|------|------|------|------|------|
|                            | 64   | 64   | 32   | 16   | 8    | 8    |
| <i>Discrete datasets</i>   |      |      |      |      |      |      |
| Conway                     | 0.0  | 0.0  | 0.0  | 0.0  | 0.0  | 0.0  |
| Chess                      | 8.9  | 25.2 | 24.4 | 15.4 | 12.3 | 10.3 |
| 2048                       | 1.8  | 4.4  | 2.9  | 1.4  | 0.9  | 0.8  |
| Fifteen Puzzle             | 26.4 | 54.9 | 48.5 | 16.8 | 12.7 | 3.4  |
| Tetris                     | 7.0  | 10.0 | 9.6  | 2.8  | 3.2  | 4.5  |
| Snake                      | 1.5  | 6.3  | 4.7  | 3.6  | 2.4  | 2.6  |
| Rubik’s Cube 3D            | 36.5 | 36.9 | 31.0 | 26.9 | 22.0 | 21.6 |
| <i>Continuous datasets</i> |      |      |      |      |      |      |
| Double Pendulum            | 1.1  | 1.1  | 1.1  | 1.1  | 1.1  | 1.1  |
| Three Body                 | 0.1  | 0.6  | 0.3  | 0.2  | 0.3  | 0.5  |
| Colliding Balls 2D         | 0.0  | 0.0  | 0.0  | 0.0  | 0.0  | 0.0  |
| Colliding Balls 3D         | 0.1  | 0.0  | 0.0  | 0.1  | 0.0  | 0.1  |
| Pong                       | 0.2  | 1.3  | 0.7  | 0.4  | 0.2  | 0.1  |

Table 6: **Token order ablation.** Zigzag and Hilbert orders offer no consistent advantage over raster order, which we use as the default.

| Block size                 | Raster | Zigzag | Hilbert |
|----------------------------|--------|--------|---------|
|                            | 8      | 8      | 8       |
| <i>Discrete datasets</i>   |        |        |         |
| Conway                     | 0.0    | 0.0    | 0.0     |
| Chess                      | 10.3   | 14.9   | 11.8    |
| 2048                       | 0.8    | 1.3    | 1.0     |
| Fifteen Puzzle             | 3.4    | 2.4    | 1.6     |
| Tetris                     | 4.5    | 6.3    | 10.3    |
| Snake                      | 2.6    | 2.4    | 4.1     |
| Rubik’s Cube 3D            | 21.6   | 21.2   | 22.8    |
| <i>Continuous datasets</i> |        |        |         |
| Double Pendulum            | 1.1    | 1.2    | 1.2     |
| Three Body                 | 0.5    | 0.4    | 0.7     |
| Colliding Balls 2D         | 0.0    | 0.0    | 0.0     |
| Colliding Balls 3D         | 0.1    | 0.1    | 0.1     |
| Pong                       | 0.1    | 0.2    | 0.3     |

default for all our datasets. See Appendix C.1 for the full sweep of context noise levels for all serial methods.

**Block size.** Table 5 shows that smaller block sizes reduce errors for discrete datasets. Continuous datasets seem less sensitive to the choice of block size. We train additional sets of models called Pin-64 with block sizes  $b \in \{64, N_v\}$  and Pin-8 with block sizes  $b \in \{8, N_v\}$ . Focusing training on a particular block size improves performance for that block size at the cost of flexibility.

**Token order.** For small block sizes, the token order determines the spatial grouping used by individual blocks. We compare Raster, Zigzag, and Hilbert orderings in Table 6. Zigzag uses an order similar to JPEG (Wallace, 1991), and Hilbert uses a 2D Hilbert curve (Hilbert, 1891) starting in the top-left corner. More elaborate token orders offer no advantage over Raster, so we use it as default.

## 6 CONCLUSION

The Serial Scaling Hypothesis (Liu et al., 2026) proved that diffusion models are not very effective at modeling serial computation. Videos are full of causal dependencies that require serial computation. Across a collection of games and physical simulations, we demonstrate that scaling training data does not enable bidirectional diffusion models to model complex video. Across many datasets and two architectures, we show serial methods outperform the parallel baseline with matched model sizes and training budgets. We introduce S2PD, a simple extension to block-causal diffusion that establishes video structure through autoregressive denoising at high noise levels before refining the details in parallel. This preserves physical and logical consistency from serial computation while improving sampling efficiency and temporal stability over other serial methods.

## AI USE STATEMENT

We used OpenAI Codex and Anthropic Claude for software implementation and debugging, literature organization, experiment planning, and manuscript editing. The authors reviewed the AI-assisted work and take responsibility for the submission.

## ACKNOWLEDGMENTS

We thank Felix O’Mahony and Yuxin Yao for reviewing the manuscript and providing helpful feedback. This work was funded in part by Toyota Motor Europe. The authors acknowledge the use of resources provided by the Isambard-AI (McIntosh-Smith et al., 2024) National AI Research Resource (AIRR). Isambard-AI is operated by the University of Bristol and is funded by the UK Government’s Department for Science, Innovation and Technology (DSIT) via UK Research and Innovation; and the Science and Technology Facilities Council [ST/AIRR/I-A-I/1023].

## REFERENCES

- Anthropic. Claude Fable 5 and Claude Mythos 5, 2026. URL <https://www.anthropic.com/news/claude-fable-5-mythos-5>.
- Marianne Arriola, Aaron Gokaslan, Justin T. Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2503.09573>.
- Hritik Bansal, Clark Peng, Yonatan Bitton, Roman Goldenberg, Aditya Grover, and Kai-Wei Chang. VideoPhy-2: A challenging action-centric physical commonsense evaluation in video generation. In *International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2503.06800>.
- Black Forest Labs. FLUX.2: Analyzing and enhancing the latent space of FLUX – representation comparison, 2025. URL <https://bfl.ai/techblog/representation-comparison>.
- Boyuan Chen, Diego Martí Monsó, Yilun Du, Max Simchowitz, Russ Tedrake, and Vincent Sitzmann. Diffusion forcing: Next-token prediction meets full-sequence diffusion. In *Advances in Neural Information Processing Systems*, 2024. URL <https://arxiv.org/abs/2407.01392>.
- Jorge Díaz Chao, Konpat Preechakul, Yuxi Liu, and Yutong Bai. The seriality gap in video diffusion models. *arXiv preprint arXiv:2607.13031*, 2026. URL <https://arxiv.org/abs/2607.13031>.
- D. C. Dowson and B. V. Landau. The Fréchet distance between multivariate normal distributions. *Journal of Multivariate Analysis*, 12(3):450–455, 1982. doi: 10.1016/0047-259X(82)90077-X.
- Martin Gardner. Mathematical games: The fantastic combinations of John Conway’s new solitaire game “life”. *Scientific American*, 223(4):120–123, 1970. doi: 10.1038/scientificamerican1070-120.
- Songwei Ge, Aniruddha Mahapatra, Gaurav Parmar, Jun-Yan Zhu, and Jia-Bin Huang. On the content bias in Fréchet video distance. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024. URL <https://arxiv.org/abs/2404.12391>.
- Google DeepMind. Genie 3: A new frontier for world models, 2025. URL <https://deepmind.google/blog/genie-3-a-new-frontier-for-world-models/>.
- Klaus Greff, Francois Belletti, Lucas Beyer, Carl Doersch, Yilun Du, Daniel Duckworth, David J. Fleet, Dan Gnanaprasagam, Florian Golemo, Charles Herrmann, Thomas Kipf, Abhijit Kundu, Dmitry Lagun, Issam Laradji, Hsueh-Ti (Derek) Liu, Henning Meyer, Yishu Miao, Derek Nowrouzezahrai, Cengiz Oztireli, Etienne Pot, Noha Radwan, Daniel Rebain, Sara Sabour, Mehdi S. M. Sajjadi, Matan Sela, Vincent Sitzmann, Austin Stone,

- Deqing Sun, Suhani Vora, Ziyu Wang, Tianhao Wu, Kwang Moo Yi, Fangcheng Zhong, and Andrea Tagliasacchi. Kubric: A scalable dataset generator. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 3749–3761, 2022. URL [https://openaccess.thecvf.com/content/CVPR2022/html/Greff\\_Kubric\\_A\\_Scalable\\_Dataset\\_Generator\\_CVPR\\_2022\\_paper.html](https://openaccess.thecvf.com/content/CVPR2022/html/Greff_Kubric_A_Scalable_Dataset_Generator_CVPR_2022_paper.html).
- Jing Gu, Xian Liu, Yu Zeng, Ashwin Nagarajan, Fangrui Zhu, Daniel Hong, Yue Fan, Qianqi Yan, Kaiwen Zhou, Ming-Yu Liu, and Xin Eric Wang. PhyWorldBench: A comprehensive evaluation of physical realism in text-to-video models. In *International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2507.13428>.
- Xuyang Guo, Jiayan Huo, Zhenmei Shi, Zhao Song, Jiahao Zhang, and Jiale Zhao. T2VPhysBench: A first-principles benchmark for physical consistency in text-to-video generation. *arXiv preprint arXiv:2505.00337*, 2025. URL <https://arxiv.org/abs/2505.00337>.
- David Hilbert. Ueber die stetige abbildung einer linie auf ein flächenstück. *Mathematische Annalen*, 38(3):459–460, 1891. doi: 10.1007/BF01199431.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2006.11239>.
- Jonathan Ho, Tim Salimans, Alexey Gritsenko, William Chan, Mohammad Norouzi, and David J. Fleet. Video diffusion models. In *Advances in Neural Information Processing Systems*, 2022. URL <https://arxiv.org/abs/2204.03458>.
- Anthony Hu, Václav Volhejn, Adrien Ramanana Rahary, Chris Mulder, Aditya Makkar, Alyx Liao, Amélie Royer, Manu Orsini, Adam Jelley, Eloí Alonso, Florian Laurent, Fredrik Norén, James Swingos, Jan Hünemann, Kent Rollins, Lucas Hosseini, Matthieu Le Cauchois, Maxim Peter, Pim de Witte, Tim Brown, Vincent Micheli, Moritz Böhle, Gabriel de Marmiesse, Viktoriia Sharmanska, Lucia Specia, Michael Black, and Patrick Pérez. Multiplayer interactive world models with representation autoencoders. *arXiv preprint arXiv:2607.05352*, 2026. URL <https://arxiv.org/abs/2607.05352>.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://arxiv.org/abs/2106.09685>.
- Xun Huang, Zhengqi Li, Guande He, Mingyuan Zhou, and Eli Shechtman. Self forcing: Bridging the train-test gap in autoregressive video diffusion. In *Advances in Neural Information Processing Systems*, 2025. URL <https://arxiv.org/abs/2506.08009>.
- Ziqi Huang, Yinan He, Jiashuo Yu, Fan Zhang, Chenyang Si, Yuming Jiang, Yuanhan Zhang, Tianxing Wu, Qingyang Jin, Nattapol Chanpaisit, Yaohui Wang, Xinyuan Chen, Limin Wang, Dahua Lin, Yu Qiao, and Ziwei Liu. VBench: Comprehensive benchmark suite for video generative models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 21807–21818, 2024. URL [https://openaccess.thecvf.com/content/CVPR2024/html/Huang\\_VBench\\_Comprehensive\\_Benchmark\\_Suite\\_for\\_Video\\_Generative\\_Models\\_CVPR\\_2024\\_paper.html](https://openaccess.thecvf.com/content/CVPR2024/html/Huang_VBench_Comprehensive_Benchmark_Suite_for_Video_Generative_Models_CVPR_2024_paper.html).
- Žiga Kovačič and Kevin Ellis. MPMWorlds: Material-point-method simulations for inferring and extrapolating physical dynamics. *arXiv preprint arXiv:2606.01538*, 2026. URL <https://arxiv.org/abs/2606.01538>.
- Tianhong Li and Kaiming He. Back to basics: Let denoising generative models denoise. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026. URL <https://arxiv.org/abs/2511.13720>.
- Juyi Lin, Arash Akbari, Yumei He, Lin Zhao, Haichao Zhang, Arman Akbari, Xingchen Xu, Zoe Y. Lu, Enfu Nan, Hokin Deng, Edmund Yeh, Sarah Ostadabbas, Yun Fu, Jennifer Dy, Pu Zhao, and Yanzhi Wang. PhyGround: Benchmarking physical reasoning in generative world models. *arXiv preprint arXiv:2605.10806*, 2026. URL <https://arxiv.org/abs/2605.10806>.

- Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2210.02747>.
- Yuxi Liu, Konpat Preechakul, Kananart Kuwarananchaoen, and Yutong Bai. The serial scaling hypothesis. In *International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2507.12549>.
- Xinyin Ma, Julius Berner, Chao Liu, Arash Vahdat, Weili Nie, and Xinchao Wang. Flex-forcing: Towards a unified autoregressive and bidirectional video diffusion model. In *International Conference on Machine Learning*, 2026. URL <https://arxiv.org/abs/2607.03509>.
- Simon McIntosh-Smith, Sadaf R. Alam, and Christopher Woods. Isambard-AI: A leadership class supercomputer optimised specifically for artificial intelligence. In *Proceedings of the Cray User Group Conference*, 2024. URL [https://www.cug.org/proceedings/cug2024\\_proceedings/includes/files/pap130s2-file1.pdf](https://www.cug.org/proceedings/cug2024_proceedings/includes/files/pap130s2-file1.pdf).
- Saman Motamed, Laura Culp, Kevin Swersky, Priyank Jaini, and Robert Geirhos. Do generative video models understand physical principles? In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pp. 948–958, 2026.
- NVIDIA. Cosmos 3: Omnimodal world models for physical ai. *arXiv preprint arXiv:2606.02800*, 2026. URL <https://arxiv.org/abs/2606.02800>.
- Junhyuk Oh, Xiaoxiao Guo, Honglak Lee, Richard L. Lewis, and Satinder Singh. Action-conditional video prediction using deep networks in Atari games. In *Advances in Neural Information Processing Systems*, 2015. URL <https://arxiv.org/abs/1507.08750>.
- Felix O’Mahony, Roberto Cipolla, and Ayush Tewari. VDAWorld: World modelling via VLM-directed abstraction and simulation. In *Proceedings of SIGGRAPH Asia*, 2026. URL <https://arxiv.org/abs/2512.11061>.
- OpenAI. GPT-6 Astra: A new generation of intelligence, 2026. URL <https://openai.com/index/gpt-6-astra/>.
- William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 4195–4205, 2023. URL <https://arxiv.org/abs/2212.09748>.
- Kiwhan Song, Boyuan Chen, Max Simchowitz, Yilun Du, Russ Tedrake, and Vincent Sitzmann. History-guided video diffusion. In *International Conference on Machine Learning*, 2025. URL <https://arxiv.org/abs/2502.06764>.
- Thomas Unterthiner, Sjoerd van Steenkiste, Karol Kurach, Raphael Marinier, Marcin Michalski, and Sylvain Gelly. Towards accurate generative models of video: A new metric and challenges. *arXiv preprint arXiv:1812.01717*, 2018. URL <https://arxiv.org/abs/1812.01717>.
- Gregory K. Wallace. The JPEG still picture compression standard. *Communications of the ACM*, 34(4):30–44, 1991. doi: 10.1145/103085.103089.
- Wan Team. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025. URL <https://arxiv.org/abs/2503.20314>.
- Wilson Yan, Yunzhi Zhang, Pieter Abbeel, and Aravind Srinivas. VideoGPT: Video generation using VQ-VAE and transformers. *arXiv preprint arXiv:2104.10157*, 2021. URL <https://arxiv.org/abs/2104.10157>.
- Shaofeng Yin, Jiaxin Ge, Zora Zhiruo Wang, Chenyang Wang, Xiuyu Li, Michael J. Black, Trevor Darrell, Angjoo Kanazawa, and Haiwen Feng. Vision-as-inverse-graphics agent via interleaved multimodal reasoning. In *European Conference on Computer Vision*, 2026. URL <https://arxiv.org/abs/2601.11109>.

Table 7: **Detailed hyperparameters for S2PD with DiT-B and Wan-LoRA-5B.** Shared entries span both columns. Architecture-specific parameters are listed in separate columns.

| Setting                    | DiT-B                                           | Wan-5B-LoRA                      |
|----------------------------|-------------------------------------------------|----------------------------------|
| <i>Architecture</i>        |                                                 |                                  |
| Depth, width, heads        | 12, 768, 12                                     | 30, 3072, 24                     |
| Tubelet size               | $4 \times 32 \times 32$ pixels                  | $1 \times 2 \times 2$ latents    |
| Objective                  | x-pred                                          | v-pred                           |
| Trainable parameters       | 132.7M                                          | 80.6M / 5.00B frozen             |
| Bottleneck dimension       | 128                                             | —                                |
| LoRA rank / alpha          | —                                               | 32 / 32                          |
| <i>Training</i>            |                                                 |                                  |
| Loss                       |                                                 | v-loss                           |
| Epochs                     |                                                 | 100                              |
| Steps per epoch            |                                                 | 1,000                            |
| Batch size                 |                                                 | 32                               |
| LR schedule                |                                                 | Cosine, $10^{-4}$ to $10^{-5}$   |
| Optimizer                  |                                                 | AdamW                            |
| Optimizer parameters       | $\beta_1 = 0.9, \beta_2 = 0.95$                 | $\beta_1 = 0.9, \beta_2 = 0.999$ |
| Weight decay               | 0                                               | 0.01                             |
| Gradient-norm clipping     |                                                 | 1.0                              |
| EMA decay                  |                                                 | 0.9996                           |
| Context noise distribution |                                                 | $\mathcal{U}(0, 1)$              |
| Target noise distribution  | Plateau logit-normal, $\mu = 0.8, \sigma = 0.8$ |                                  |
| Sequence length            | $64 + 2 \times 3,840 = 7,744$ tokens            |                                  |
| Block sizes                | 4, 8, 16, 32, 64, 3,840                         |                                  |
| <i>Sampling</i>            |                                                 |                                  |
| Transition noise level     |                                                 | $\tau = 0.6$                     |
| Solver                     |                                                 | Euler, 10 steps                  |
| Block size                 | 8                                               | 64                               |

## A IMPLEMENTATION DETAILS

Table 7 summarizes architectural, training, and sampling hyperparameters for DiT-B and Wan-5B-LoRA models. For DiT-B, we use pixel-space diffusion with  $x$ -prediction and  $v$ -loss. For Wan-5B-LoRA, we use latent-space diffusion with  $v$ -prediction and  $v$ -loss. Both architectures encode text prompts with Wan’s frozen UMT5-XXL encoder. We project text token embeddings to the transformer’s width and feed them into each transformer block via cross-attention. We use a fixed prompt for each dataset except MPMWorlds. There, we use scene descriptions provided by the dataset. Training DiT-B takes about 12 hours on a single node with four H100 GPUs. Training Wan-5B-LoRA takes about 48 hours on four nodes with four H100 GPUs each. We train for 100 epochs with 1,000 steps per epoch. To standardize training across finite and procedurally generated datasets, we define one epoch as 1,000 optimizer steps and use batch sizes of 32 or 64, depending on the method. For finite datasets, we repeat samples to fill epochs. We evaluate EMA checkpoints on validation splits. For datasets that are simulated on-the-fly, we use dedicated validation seeds that are distinct from the training seeds.

## B DATASET DETAILS

Let  $s_{t-1}$  and  $s_t$  be consecutive parsed states and  $\mathcal{V}(s_{t-1})$  the permitted next states. We define the transition residual as

$$r_t = \min_{\hat{s} \in \mathcal{V}(s_{t-1})} d(\hat{s}, s_t), \quad (2)$$

where  $d$  measures how far the observed state  $s_t$  is from a permitted state  $\hat{s}$ .

**Discrete datasets.** A transition is invalid when either state breaks the dataset’s state rules or no permitted move connects them. We report the number of invalid transitions as *invalid transitions per rollout*. A move is required between gameplay states. Persistent state errors are charged on every transition they affect.

**Continuous datasets.** We extrapolate each frame from past frames using the simulator’s dynamics and compare that predicted extrapolation with what we observe in the generated video. For each object, let  $P_{t,i}$  and  $O_{t,i}$  be its predicted and observed regions at time  $t$ . Its residual is  $1 - \text{IoU}(P_{t,i}, O_{t,i})$  after a small positional tolerance for parsing error. We average the residual over the source’s objects and report the sum over frames as *dynamics error per rollout*. An expected object that is missing counts as a full residual error for each frame that it is missing.

### B.1 PER-DATASET DETAILS

We describe the dataset-specific evaluation rules below. The released code provides the full parsing procedures and details.

**Conway.** Each grid must follow from the previous grid under Conway’s rules.

**Chess.** Each side must have one king, and pawns cannot stand on the first or last rank. We infer the moving side from the first valid transition, then enforce alternating turns. We infer possible castling and en-passant rights from the visible board. Moving a pinned piece is illegal. A win modal must follow a visible checkmate. We accept a draw modal after any valid board without checkmate. An invalid modal counts as one error when it appears.

**2048.** Each transition must consist of a horizontal or vertical slide that changes the board, followed by the appearance of a new 2 or 4 tile in an empty cell.

**Fifteen Puzzle.** Each board must contain every numbered tile and one blank exactly once. A move slides a tile adjacent to the blank into that space.

**Tetris.** The previewed piece must drop vertically into a column and reach its resting position when the board has no full rows. When the board contains full rows, the next transition must clear them and shift the rows above downward, leaving the preview unchanged. We allow any rotation of the dropped piece, and the preview after a drop may show any piece.

**Snake.** A valid board contains one connected chain from head to tail and one food cell. On each move, the head advances to a free neighboring cell and the tail moves forward. If the head reaches food, the snake grows instead, and new food appears in an empty cell. An end-of-game modal is valid only when the preceding board is valid and the head is trapped.

**Rubik’s Cube 3D.** We fit a cube to its silhouette and parse the colors of the visible facelets. The silhouette determines the camera orientation only up to a quarter turn. We therefore test the possible camera orientations and legal quarter turns, accepting a transition if one combination matches every facelet visible in both frames. We do not enforce color counts or solvability, and hidden colors remain unconstrained. The metric thus checks local motion rather than persistent identity through occlusion.

**Colliding Balls 2D.** We track balls by color and predict their motion using the simulator’s contact law, allowing collisions to occur between frames. We penalize new overlaps between balls and new penetrations of the table rails. Since balls cannot enter or leave the table, each unmatched arrival or departure incurs one full ball penalty.

**Colliding Balls 3D.** We estimate the camera pose from the table and apply the 2D checks in table coordinates. An arrival or departure is not penalized when a nearby visible ball could explain it through occlusion, or when the arriving or departing ball touches the image border. Each visible ball can account for at most one arrival and one departure per frame. We exclude balls clipped by the border from motion scoring and charge a failed table fit once when it begins.

**Double Pendulum.** We fit rod lengths and the mass ratio once per rollout, then use the pendulum equations to predict motion. Observed rod lengths must remain close to their rollout medians. Unexplained missing bobs incur a full-frame penalty. Only the second bob may leave the image without penalty. Its last observations must place it near the edge or moving beyond it, and its rod must be long enough to reach outside the image.

**Three Body.** We fit gravity once per rollout and predict each body’s motion from the forces exerted by visible bodies. When recent motion indicates that a body has left the image or become occluded, we do not penalize its absence until it returns. A body missing from the start is penalized on every frame in which it is absent.

**Pong.** We predict ball motion using the simulator’s wall and paddle collisions. Paddles must remain on their rails and inside the court. After crossing a goal line, the ball may reappear at the serve position. Other missing or extra objects incur a penalty on every affected frame.

## C ADDITIONAL QUANTITATIVE RESULTS

Table 8: **CD-FVD ↓ with Wan-5B-LoRA at block size 64.** BC and cDF vary the re-noising used to resist autoregressive drift and Ours varies the transition noise level.

|                   | BC    |       |       |       |       | cDF    |       |       |       |       | Ours  |       |       |       |       |
|-------------------|-------|-------|-------|-------|-------|--------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
|                   | 0.0   | 0.2   | 0.4   | 0.6   | 0.8   | 0.0    | 0.2   | 0.4   | 0.6   | 0.8   | 0.0   | 0.2   | 0.4   | 0.6   | 0.8   |
| Kubric MOVi-A     | 176.1 | 141.2 | 153.8 | 182.6 | 278.7 | 366.2  | 168.5 | 165.8 | 175.5 | 240.0 | 162.1 | 144.6 | 139.2 | 137.0 | 137.7 |
| Kubric MOVi-C     | 265.5 | 240.3 | 286.8 | 469.9 | 913.0 | 1353.9 | 376.5 | 345.4 | 454.1 | 894.9 | 319.9 | 250.4 | 227.0 | 231.7 | 271.3 |
| MPMWorlds         | 48.2  | 43.9  | 49.0  | 63.3  | 115.5 | 101.1  | 43.2  | 47.3  | 58.8  | 95.5  | 50.6  | 41.8  | 42.1  | 42.5  | 43.1  |
| Rubik’s Cube Real | 69.4  | 70.9  | 68.2  | 68.3  | 66.9  | 96.2   | 70.8  | 71.9  | 71.8  | 74.9  | 68.6  | 66.6  | 65.6  | 64.4  | 65.0  |

Table 9: **FVD ↓ with Wan-5B-LoRA at block size 64.** BC and cDF vary the re-noising used to resist autoregressive drift and Ours varies the transition noise level.

|                   | BC    |       |       |       |       | cDF    |       |       |       |       | Ours  |       |       |       |       |
|-------------------|-------|-------|-------|-------|-------|--------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
|                   | 0.0   | 0.2   | 0.4   | 0.6   | 0.8   | 0.0    | 0.2   | 0.4   | 0.6   | 0.8   | 0.0   | 0.2   | 0.4   | 0.6   | 0.8   |
| Kubric MOVi-A     | 241.0 | 186.1 | 210.4 | 202.2 | 234.0 | 582.1  | 249.3 | 242.7 | 266.2 | 254.7 | 221.3 | 176.0 | 181.0 | 162.2 | 171.1 |
| Kubric MOVi-C     | 264.7 | 242.6 | 212.6 | 236.8 | 311.1 | 1115.3 | 302.7 | 231.1 | 255.9 | 333.2 | 289.3 | 228.1 | 199.0 | 186.2 | 214.1 |
| MPMWorlds         | 35.7  | 32.0  | 33.3  | 40.0  | 56.1  | 141.8  | 33.2  | 32.6  | 37.8  | 49.9  | 43.6  | 32.9  | 31.1  | 33.2  | 33.3  |
| Rubik’s Cube Real | 105.8 | 100.3 | 95.2  | 91.6  | 89.6  | 121.9  | 95.5  | 94.2  | 90.8  | 90.4  | 110.6 | 102.3 | 102.8 | 100.4 | 99.6  |

### C.1 CONTEXT NOISE LEVEL

All autoregressive diffusion methods require re-noising of generated outputs to mitigate autoregressive drift. We sweep the context noise level  $t_c$  for BC and cDF and the transition noise level  $\tau$  for S2PD from 0.0 to 1.0 in increments of 0.2, with matched block sizes. Table 8 shows that  $t_c = 0.2$  works best for BC and cDF and  $\tau = 0.6$  works best for S2PD. S2PD achieves the lowest score across all methods.

### C.2 FVD

Table 9 reports FVD scores, which broadly agree with the CD-FVD scores. S2PD achieves the lowest FVD on both Kubric datasets and the MPMWorlds dataset. However, cDF and BC outperform our method on the Rubik’s Cube Real dataset. We attribute this to the limited diversity of the real-world dataset. For both FVD and CD-FVD, we exclude the condition image and compare features from temporally subsampled 16-frame windows.

### C.3 MPMWORLDS

We obtained the MPMWorlds dataset from its authors (Kovačič & Ellis, 2026). We re-time all video from 30 fps to 24 fps without dropping frames. Their evaluation code was not available at the time of our experiments, so we could not run their exact metrics. However, we implemented our own versions of the metrics based on their paper. Table 10 reports these metrics for all four of our methods. These metrics assess foreground overlap, object collapse, motion, color, and temporal consistency. Unfortunately, these metrics depend strongly on pixel-wise alignment with the ground-truth reference video.

We used a short text description rather than code conditioning to keep the setup consistent with the other datasets. Generated samples are not expected to match their corresponding ground truth videos exactly. This is especially true for MPMWorlds as physical parameters such as rigidity and viscosity

Table 10: **Reimplemented MPMWorlds metrics.** Unofficial metrics implemented based on the paper.

| Metric                  | Parallel      |      | Serial |      |
|-------------------------|---------------|------|--------|------|
|                         | Bidirectional | cDF  | BC     | Ours |
| Foreground mIoU ↑       | 0.47          | 0.45 | 0.45   | 0.45 |
| Object collapse ↓       | 0.80          | 0.72 | 0.71   | 0.70 |
| Motion-activity error ↓ | 0.37          | 0.34 | 0.33   | 0.34 |
| Color total variation ↓ | 0.38          | 0.31 | 0.30   | 0.30 |
| Temporal anomaly ↓      | 0.07          | 0.10 | 0.12   | 0.10 |
| Mean composite error ↓  | 0.43          | 0.40 | 0.40   | 0.40 |

cannot be inferred from the initial image alone, yet significantly affect how the scene will evolve. This ambiguity renders these metrics useless to us, but strong code conditioning is an interesting direction for future work.

## D ADDITIONAL QUALITATIVE RESULTS

Here we show frames from videos generated by our method and the bidirectional baseline for each dataset. In Figure 8, the x axis shows step count for each game. Each step spans four frames at 24 frames per second, so steps 0–8 show selected frames from the first 1.33 seconds of each 10-second sample. We exclude intermediate animation frames and show consecutive settled states—the same states parsed for evaluation. Figures 9 and 10 show one frame per second for the first eight seconds of each 10-second sample. The first frame of each row is the conditioning image.

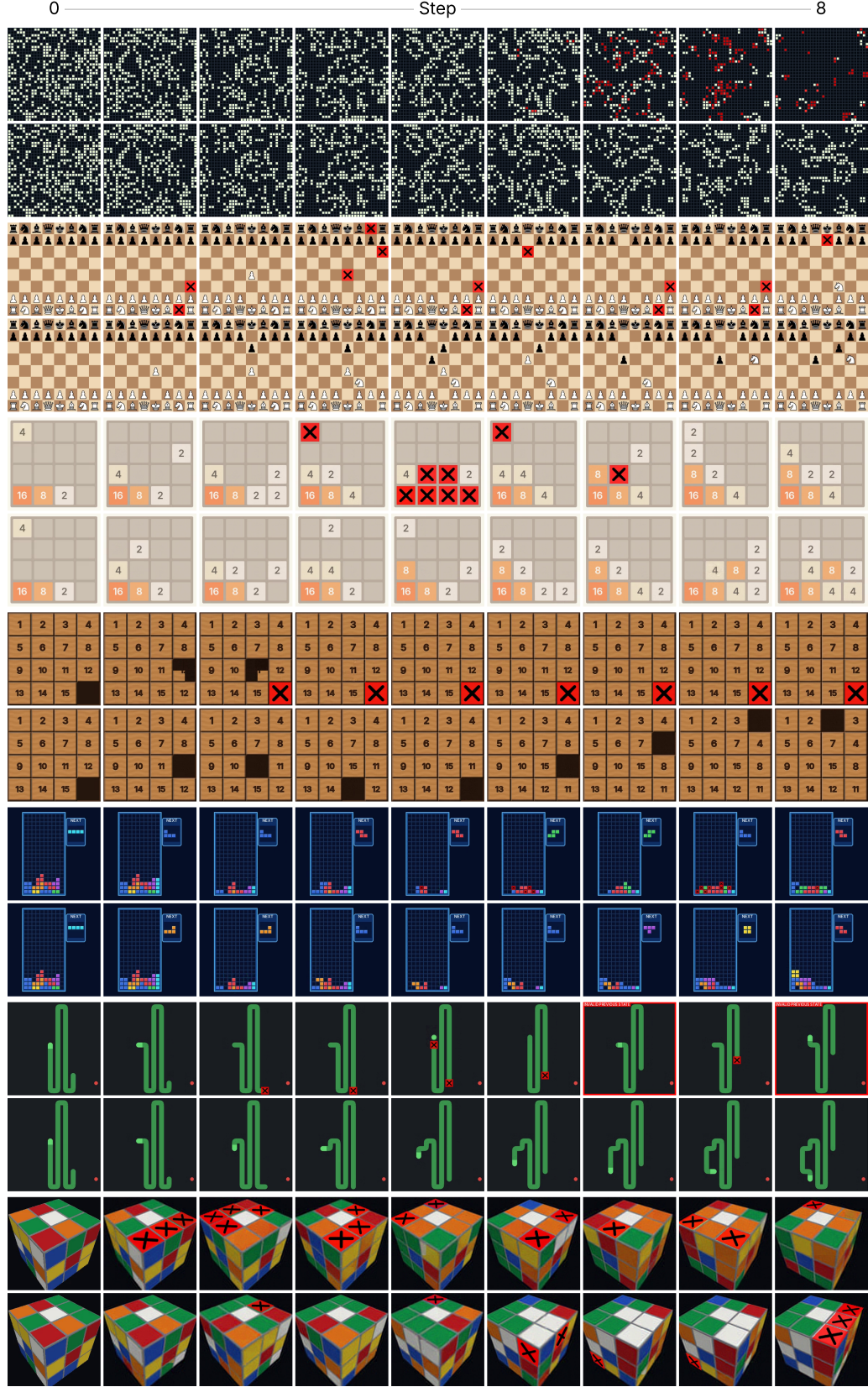


Figure 8: **Discrete datasets.** Qualitative comparison on Conway, Chess, 2048, Fifteen Puzzle, Tetris, Snake, and Rubik's Cube 3D. For each dataset, the top row shows Bidirectional and the bottom row shows Ours. We show select frames from the ten-second video samples.

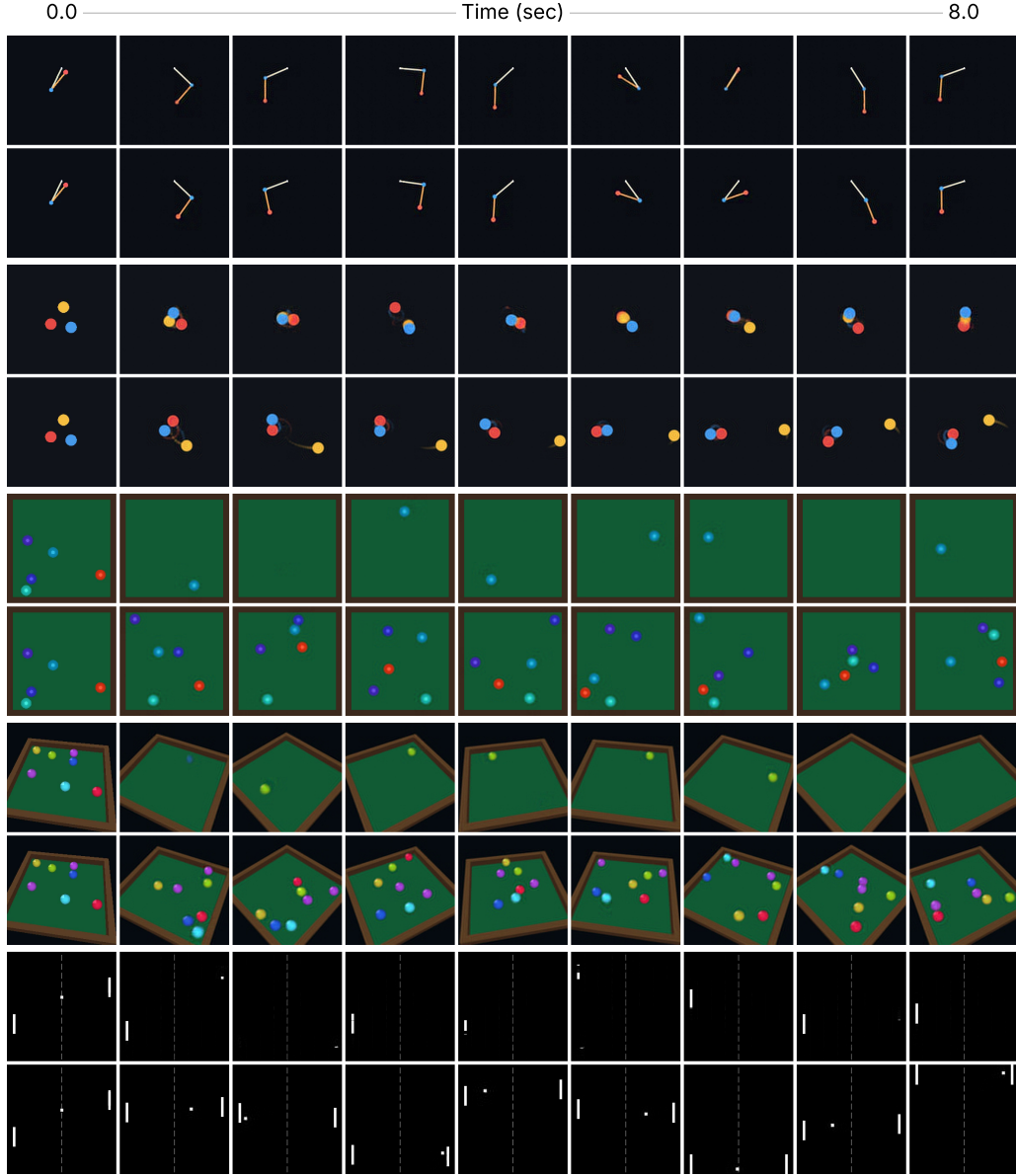


Figure 9: **Continuous datasets.** Qualitative comparison on Double Pendulum, Three Body, Colliding Balls 2D, Colliding Balls 3D, and Pong. For each dataset, the top row shows Bidirectional and the bottom row shows Ours. We show select frames from the ten-second video samples.

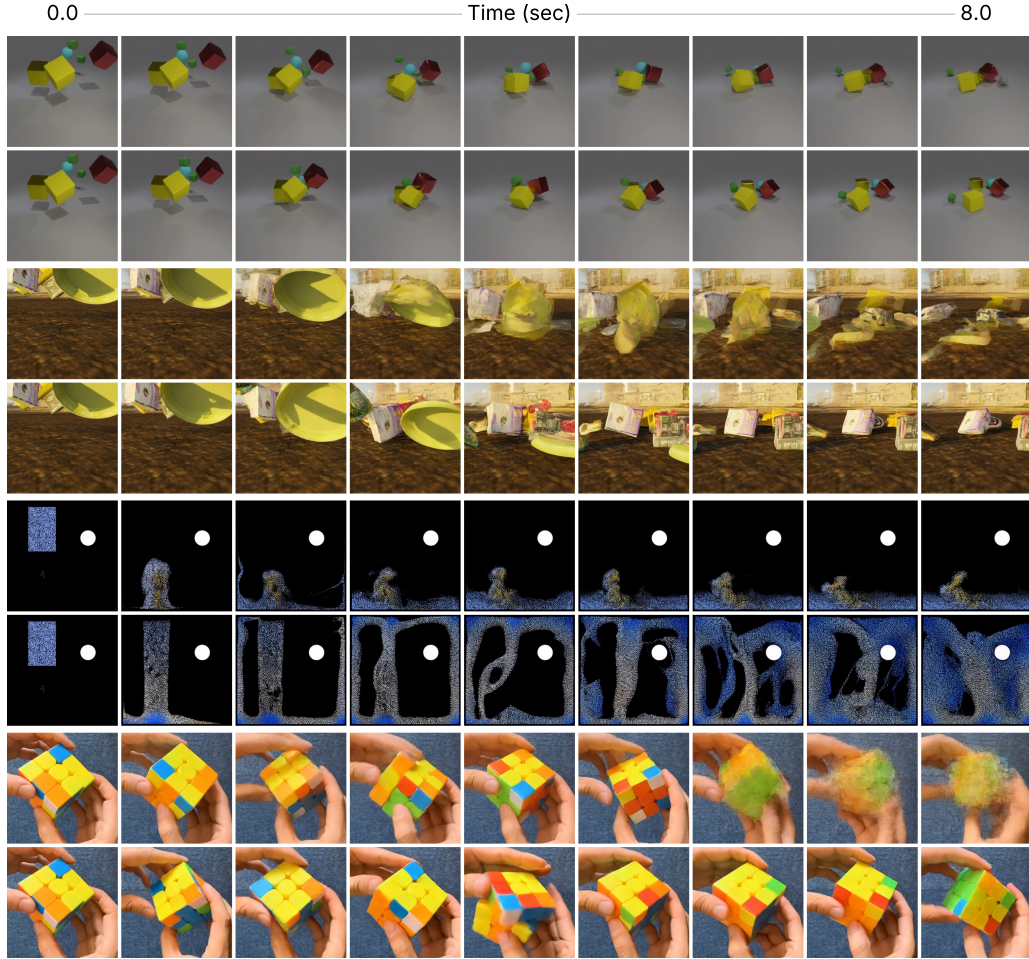


Figure 10: **Video datasets.** Qualitative comparison on Kubric MOVi-A, Kubric MOVi-C, MPM-Worlds, and Rubik’s Cube Real. For each dataset, the top row shows Bidirectional and the bottom row shows Ours. We show select frames from the ten-second video samples.